

User Guide

What is this system?

This is an MCQ (Multiple Choice Question) knowledge base generator built for Data Science, Machine Learning, and AI interview preparation. You feed it sources -- URLs, PDFs, or local files -- and it produces quality-validated MCQs in a structured Excel spreadsheet. The questions are classified by category, topic, and difficulty, and validated against a 5-point quality checklist used for FAANG-level interview prep.

Prerequisites

- **Python 3.10+** installed on your machine
- **An API key** from a supported LLM provider (free options available)
- **Git** (optional, for cloning)

Getting an API key (2 minutes)

The recommended provider is **Google Gemini** -- it is completely free and requires no credit card.

1. Go to <https://makersuite.google.com/app/apikey>
2. Click "Get API Key"
3. Copy the key (starts with **AIza...**)

That is it. You are ready to install.

Installation

```
# Clone the repo (or unzip if you received it as a zip)
git clone <repo-url>
cd DesignYourAI

# Create and activate virtual environment
python -m venv .venv

# Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# Windows Command Prompt:
.venv\Scripts\activate.bat

# macOS/Linux:
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

```
# Install Playwright browsers (needed for JS-heavy websites)
playwright install chromium
```

Configure your API key

Create a file called `.env` in the project root (or copy from `.env.template`):

```
LLM_PROVIDER=gemini
GEMINI_API_KEY=AIza...your-key-here
```

Verify setup

```
python test_sample.py
```

If this prints "All tests passed" you are good to go.

How the engine works

When you give the engine a source, it goes through a 6-step pipeline:

```
Source (URL / PDF / file)
|
v
[1] FETCH -- download the content (static HTML, browser render, or file read)
|
v
[2] DETECT SHAPE -- what kind of content is this?
|
+-- Already MCQ? (has A/B/C/D options + answer) --> direct parse
+-- Q&A pairs? (question + answer text)           --> convert to MCQ
+-- Prose? (article, tutorial, docs)              --> generate MCQs from it
|
v
[3] CLASSIFY -- assign Category, Topic, and Difficulty
|   Category:   Conceptual (50%) / Mathematical (25%) / Application (25%)
|   Topic:      AI / ML / Data Science / System Design
|   Difficulty: Easy / Medium / Hard
|
v
[4] DEDUPLICATE -- remove questions that are identical or very similar
|   Uses hash matching (exact) + fuzzy matching (85% similarity)
|
v
[5] QUALITY VALIDATE -- 5-point MAANG checklist
|   1. Tests reasoning, not just recall
|   2. Plausible distractors (wrong options look reasonable)
```

```
| 3. Options are clearly differentiated
| 4. Logical structure
| 5. Industry-level rigor
|
v
[6] STORE -- write to Excel with row-by-row fallback
      If one row fails (e.g. illegal characters), only that row
      is skipped. The rest of the batch is saved.
```

You do not need to manage any of these steps. They run automatically.

Source reliability guide

Not all sources work equally well. Here is an honest breakdown based on real testing:

Source type	Reliability	Typical yield	How it works	Examples
GitHub repos (Q&A markdown)	~95%	High (50-300 Qs per repo)	Parses Q&A headings directly, converts to MCQ	kojino/120-DS, youssefHosni/DS-Q&A, andrewekhalel/MLQuestions
PDFs / DOCX (local files)	~95%	Depends on file	Extracts text, parses or generates	Any downloaded or exported file
Technical blogs / Q&A sites	~75%	Medium (5-30 per page)	Renders with browser, generates MCQs from prose	GeeksforGeeks, Guru99, Wikipedia, company engineering blogs
Structured quiz sites	~30%	Low (0-5 per page)	Tries to parse DOM for quiz markup	Simplilearn, InterviewBit free tier
Bot-protected sites (Cloudflare)	0%	None	Blocked before content loads	Sanfoundry, some enterprise sites
Login-gated sites	0%	None	Cannot authenticate	madinterview, LeetCode premium, Educative

What to do when a source does not work

1. Run `--dry-run` first to see if the engine can fetch content
2. If 0 words fetched or "blocked" message: the site is protected
3. **Workaround:** Open the page in your browser, press Ctrl+P, save as PDF, then use `--pdf`

This workaround works for any content you can see in your browser, including login-gated and bot-protected sites.

Using `run_ingest.py` (primary CLI)

This is the main entry point. One command, one source, automatic routing.

Ingest from a URL

```
# GitHub repo (highest reliability)
python run_ingest.py --url https://github.com/kojino/120-Data-Science-Interview-Questions

# Technical blog
python run_ingest.py --url https://www.geeksforgeeks.org/machine-learning/machine-learning-interview-questions/

# Wikipedia article
python run_ingest.py --url https://en.wikipedia.org/wiki/Gradient_descent
```

Ingest from a PDF

```
python run_ingest.py --pdf data/raw/interview-prep.pdf
python run_ingest.py --pdf ~/Downloads/ml-questions.pdf
```

Ingest from a local file

```
python run_ingest.py --file notes.md
python run_ingest.py --file interview-questions.txt
python run_ingest.py --file report.docx
```

Batch mode (multiple sources at once)

Create a text file with one source per line:

```
# sources.txt
https://github.com/kojino/120-Data-Science-Interview-Questions
https://github.com/andrewekhalel/MLQuestions
data/raw/my-notes.pdf
```

Then run:

```
python run_ingest.py --batch sources.txt
```

Lines starting with `#` are treated as comments and skipped.

Dry run (preview without processing)

Use `--dry-run` to see what the engine would do without calling the LLM or writing to Excel:

```
python run_ingest.py --url https://some-site.com/questions --dry-run
```

Output looks like:

```
[1/1] https://some-site.com/questions  
[ok] fetched 3200 words  
-> shape: QA (45 question markers)
```

This tells you:

- Whether the fetch succeeded
- How much content was found
- What shape was detected (MCQ / QA / PROSE)

Always dry-run first when trying a new source. It costs nothing and saves LLM tokens.

Limit output

Cap the number of MCQs processed per source:

```
python run_ingest.py --url https://github.com/youssefHosni/Data-Science-Interview-Questions-Answers --limit 20
```

Useful when:

- Testing a new source for the first time
- Conserving GPU/API usage
- Your local Ollama model is running hot

Custom output path

```
python run_ingest.py --url https://... --output data/knowledge_base/custom_kb.xlsx
```

Default output: `data/knowledge_base/mcq_knowledge_base.xlsx`

Using `main.py` (original CLI)

The original entry point. Works the same way but accepts multiple sources and a target count:

```
# Single source
python main.py --sources "https://example.com/questions"

# Multiple sources
python main.py --sources "https://site1.com" "data/raw/file.pdf"

# Batch file
python main.py --batch sources.txt --target 1000
```

When to use `main.py` vs `run_ingest.py`:

- `run_ingest.py` has `--dry-run`, `--limit`, and `--output`. Use it for daily work.
- `main.py` has `--target` (sets a KB size goal). Use it when building towards a specific count.

Both use the same pipeline underneath. No difference in output quality.

Using `run_stratascratch.py` (StrataScratch API)

A dedicated runner for StrataScratch's GraphQL API. This is separate because it uses an API, not web scraping.

Setup:

1. Get a StrataScratch auth token (from browser DevTools while logged in)
2. Add to `.env`:

```
STRATASCRATCH_TOKEN=your-token-here
```

Run:

```
python run_stratascratch.py
```

This fetches Q&A pairs with metadata (company name, job roles, difficulty) and converts them to MCQs. Results include the Company and Job_Roles columns that other sources leave blank.

Understanding the output

All output goes to an Excel file (default: `data/knowledge_base/mcq_knowledge_base.xlsx`).

Column reference

Column	Description	Example
Question_ID	Unique UUID	a3b2c1d4- ...

Column	Description	Example
Question_Text	The question	"Which regularization technique adds an L1 penalty...?"
Option_A	First option	"Ridge regression"
Option_B	Second option	"Lasso regression"
Option_C	Third option	"Elastic net"
Option_D	Fourth option	"Polynomial regression"
Correct_Answer	A, B, C, or D	"B"
Explanation	Why this answer is correct	"Lasso uses L1 penalty which..."
Category	Conceptual / Mathematical / Application	"Conceptual"
Topic	AI / ML / Data Science / System Design	"ML"
Difficulty	Easy / Medium / Hard	"Medium"
Source	Where this MCQ came from	"https://github.com/kojino/..."
Date_Added	When it was ingested	"2026-04-17"
Used_Status	Has it been used for posting?	FALSE
Company	Company associated with question	"Google" (may be blank)
Job_Roles	Relevant job roles	"Data Scientist, ML Engineer" (may be blank)
Image_Path	Local path to image if any	Usually blank
Image_URL	Source URL of image if any	Usually blank
Has_Image	Whether question has an image	FALSE
Image_Format	Image file format	Usually blank

What "good" output looks like

- All rows have Question_Text, all 4 options, and an Explanation filled
- Correct_Answer is always one of A, B, C, D
- Category distribution is roughly 50% Conceptual / 25% Mathematical / 25% Application
- No duplicate questions (check Source column for variety)
- Difficulty is distributed across Easy, Medium, Hard

How to spot bad rows

- Explanation is empty or very short (under 10 words)
- Multiple options say the same thing
- Question is too vague ("What is ML?") or too specific ("What was the learning rate in paper X?")

- "nan" appears as text in any cell (should not happen, but check after first run)

LLM provider options

The engine supports multiple LLM providers. You can switch at any time by changing `.env`.

Provider	Cost	Speed	Quality	Setup
Gemini (recommended)	Free	Fast	Good	<code>LLM_PROVIDER=gemini</code> + <code>GEMINI_API_KEY=...</code>
Ollama (local)	Free	Depends on GPU	Good with 7B+ models	<code>LLM_PROVIDER=ollama</code> (runs on your machine)
Groq	Free tier	Very fast	Good	<code>LLM_PROVIDER=groq</code> + <code>GROQ_API_KEY=...</code>
Together.ai	Free credits	Fast	Good	<code>LLM_PROVIDER=together</code> + <code>TOGETHER_API_KEY=...</code>
Qwen	Free tier	Medium	Good	<code>LLM_PROVIDER=qwen</code> + <code>QWEN_API_KEY=...</code>
DeepSeek	Free tier	Medium	Good	<code>LLM_PROVIDER=deepseek</code> + <code>DEEPSEEK_API_KEY=...</code>
OpenAI	Paid	Fast	Very good	<code>LLM_PROVIDER=openai</code> + <code>OPENAI_API_KEY=...</code>
Anthropic	Paid	Fast	Very good	<code>LLM_PROVIDER=anthropic</code> + <code>ANTHROPIC_API_KEY=...</code>

Switching providers

Edit `.env`:

```
LLM_PROVIDER=groq
GROQ_API_KEY=gsk_...your-key-here
```

No code changes needed. Restart the script.

Using Ollama (local, free, unlimited)

If you have a GPU (even 8GB VRAM is enough):

1. Install Ollama: <https://ollama.ai>
2. Pull a model: `ollama pull qwen2.5:7b`
3. Set in `.env`:

```
LLM_PROVIDER=ollama
```

4. Ollama runs on localhost, no API key needed.

Tradeoff: Slower than cloud providers and uses your GPU. If your GPU runs hot, take 5-10 minute breaks between runs, or switch to Gemini (free, runs on Google's servers).

Best practices for building a large knowledge base

Recommended ingestion order

1. **Start with GitHub repos** -- highest reliability, most questions, fastest.

```
python run_ingest.py --url https://github.com/youssefHosni/Data-Science-Interview-Questions-Answers
python run_ingest.py --url https://github.com/kojino/120-Data-Science-Interview-Questions
python run_ingest.py --url https://github.com/andrewekhalel/MLQuestions
```

2. **Add PDFs you already have** -- course notes, exported prep docs, downloaded guides.

```
python run_ingest.py --pdf ~/Downloads/ml-interview-guide.pdf
```

3. **Supplement with public URLs** -- GeeksforGeeks, Wikipedia, technical blogs.

```
python run_ingest.py --url https://www.geeksforgeeks.org/machine-learning/machine-learning-interview-questions/
```

4. **Fill gaps with StrataScratch** (if you have a token).

```
python run_stratascratch.py
```

Avoiding duplicates

The engine has built-in deduplication (exact hash match + 85% fuzzy similarity). You do not need to worry about running the same source twice -- duplicates are automatically dropped.

However, questions from different sources that cover the same concept but are worded differently will both be kept. This is by design -- different phrasings of the same concept are valuable for practice.

Managing GPU usage with Ollama

If using a local model:

- Use `--limit 20` for initial test runs
- Take 5-10 minute breaks between large runs (100+ questions)
- Monitor GPU temperature. If it stays above 85C for extended periods, reduce batch sizes or switch to Gemini
- Gemini is the recommended alternative -- free, fast, and does not use your hardware

Checking your progress

Open `data/knowledge_base/mcq_knowledge_base.xlsx` in Excel or any spreadsheet app at any time. The file is updated incrementally -- you can check it while the engine is running (but do not edit it while a run is in progress).

Glossary

Term	Meaning
MCQ	Multiple Choice Question -- a question with 4 options (A/B/C/D) and one correct answer
Q&A	Question and Answer pair -- a question followed by a text answer, no options
Prose	Plain text content like articles, tutorials, or documentation
Shape detection	The process of determining whether content is MCQ, Q&A, or prose
Tier 1	Static HTML extraction using BeautifulSoup (fastest, works on simple pages)
Tier 2	Browser-based extraction using Playwright (handles JavaScript-rendered sites)
Tier 3	LLM-based MCQ generation from prose content (works on anything with enough text)
MAANG-level	Quality standard modeled after Meta/Apple/Amazon/Netflix/Google interview prep
Conceptual	Questions about theory, trade-offs, comparisons, "why" and "when"
Mathematical	Questions involving formulas, probability, statistics, quantitative reasoning
Application	Questions about real-world scenarios, production systems, metric trade-offs
Deduplication	Removing identical or near-identical questions
Fuzzy matching	Detecting questions that are similar but not identical (85% threshold)
Row-by-row fallback	When saving fails on one row (e.g. illegal characters), the engine retries each row individually so only the bad row is lost, not the whole batch
Dry run	A preview run that fetches and analyzes content without calling the LLM or writing to Excel
Knowledge base (KB)	The Excel file containing all generated MCQs